C PROGRAMMING

Lesson 5 — Accepting User Input

Based on: Bro Code C Tutorial Series

1. Always Initialize Your Variables

Before accepting input, you need to declare variables to store the data. A critical rule in C is to always initialize variables when you declare them. Uninitialized variables contain random garbage values left over in memory.

```
// Bad practice - uninitialized (contains garbage values)
int age;
float gpa;

// Good practice - always initialize
int age = 0;
float gpa = 0.0;
char grade = '\0';    // null character
char name[30] = "";   // empty string, max 30 characters
```

> **Warning:** Printing an uninitialized variable gives random output every time the program runs. Always initialize to a safe default value.

2. Accepting Input with scanf()

scanf() is the standard function for reading user input in C. It works similarly to printf() but in reverse — instead of printing, it reads from the keyboard.

```
#include <stdio.h>

int main() {
    int age = 0;
    float gpa = 0.0;

    printf("Enter your age: ");
    scanf("%d", &age);       // & is the address-of operator

    printf("Enter your GPA: ");
    scanf("%f", &gpa);

    printf("Age: %d\n", age);
    printf("GPA: %.2f\n", gpa);
```

```
    return 0;
}
```

- **The & (address-of) operator** tells scanf where in memory to store the value
- **Format specifiers** are the same as printf: %d, %f, %c, %s
- **Strings (char arrays)** do NOT need & because the array name is already an address

> **Key Rule:** Always use & before variable names in scanf — except for strings (char arrays).

3. The Input Buffer Problem with %c

This is one of the most common bugs for C beginners. When you read a number with scanf(), a newline character (\n) is left behind in the input buffer. If you then try to read a char, scanf reads that leftover newline instead of waiting for your input.

```
// WRONG - grade gets the leftover \n, not your actual input
scanf("%d", &age);
scanf("%c", &grade);    // reads '\n' not the user's character!

// CORRECT - add a space before %c to skip whitespace
scanf("%d", &age);
scanf(" %c", &grade);   // the space flushes the leftover \n
```

> **Fix:** Always add a space before %c in scanf like this: scanf(" %c", &grade); — that single space skips any leftover whitespace in the buffer.

4. Reading Strings

scanf with %s — single word only

scanf("%s") stops reading at the first space. So it cannot capture full names like "John Smith".

```
char name[30] = "";
scanf("%s", name);   // reads "John" but stops at the space
                     // "Smith" stays in the buffer
```

fgets — full line including spaces

Use fgets() when you need to read a full name or sentence with spaces.

```
#include <stdio.h>
#include <string.h>   // needed for strlen()

char name[30] = "";

getchar();                          // clear leftover \n from buffer
fgets(name, sizeof(name), stdin);   // reads full line including
spaces
```

```
// fgets keeps the \n at the end — remove it
name[strlen(name) - 1] = '\0';
```

- **fgets(name, sizeof(name), stdin) —** reads a full line from keyboard
- **sizeof(name)** prevents reading more characters than the array can hold
- **fgets keeps the \n** at the end — you must manually remove it
- **getchar()** before fgets clears any leftover newline in the buffer

> **Remember:** Include #include <string.h> at the top whenever you use strlen().

5. Practice Problem

Practice: Student Registration Form

Write a C program that acts as a simple student registration form. It should ask the user for input and then print a summary.

Requirements:

- Ask for: full name (with spaces), age, GPA, and grade
- Use fgets for the name and scanf for the rest
- Handle the input buffer correctly (space before %c, getchar() before fgets)
- Print a formatted summary at the end

Expected Output

```
Enter your full name: John Smith
Enter your age: 20
Enter your GPA: 3.85
Enter your grade: A

--- Student Profile ---
Name:  John Smith
Age:   20
GPA:   3.85
Grade: A
```

> **Hint:** The trickiest part is the order of inputs. Read age and GPA first with scanf, then grade with scanf(" %c"), then use getchar() before fgets for the name.